



HASHDIT

HashDit

Smart Contract Security

ListaDao Brokered Interest Locking Mechanism

Security Audit Report

02 Jun 2026 - 11 Jun 2026

Disclaimer

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

Audit Overview

Audit Period	02 Jun 2026 – 11 Jun 2026
Overall Risk	[Medium
Project Name	ListaDao
Github	https://github.com/lista-dao/moolah/pull/182
Branch	feature/broker-interest-lock-buffer

Smart Contract List

No.	Contract Name	Verdict	Details
1	BrokerInterestLockBuffer.sol	Medium	[M01]
2	MoolahVault.sol	Low	[L01] [L02] [L03] [L04]
3	BrokerInterestRelayer.sol	Green	
4	CreditBrokerInterestRelayer.sol	Green	
5	deploy_brokerInterestLockBuffer.s .sol	Informational	[I01]

Findings

[M01] Interest Delivery, Penalty Payments, Bad Debt Recovery, and Liquidations Blocked When Vault Is Underwater	
Contract	BrokerInterestLockBuffer.sol, BrokerInterestRelayer.sol, CreditBrokerInterestRelayer.sol, LendingBroker.sol
Severity	Medium
Description	Description notifyBrokerInterest uses the buffer-adjusted vault.totalAssets() as an upper bound. When the vault is underwater (currentLocked > rawAssets, totalAssets() = 0), any non-zero amount triggers a revert, rolling back the entire supplyToVault transaction. The underwater state arises when a large interest flush sets a high currentLocked, after which significant user redemptions reduce rawAssets below currentLocked.
	<pre>JavaScript // BrokerInterestLockBuffer - notifyBrokerInterest function notifyBrokerInterest(uint256 amount) external override onlyRole(RELAYER) { if (amount > IERC4626(vault).totalAssets()) revert AmountExceedsVaultAssets(); // equivalent to: if (currentLocked() + amount > rawAssets_after) revert ... } // BrokerInterestRelayer / CreditBrokerInterestRelayer - supplyToVault MOOLAH.supply(..., balance, 0, vault, ""); address buf = IMoolahVault(vault).lockBuffer(); if (buf != address(0)) IBrokerInterestLockBuffer(buf).notifyBrokerInterest(balance);</pre>
	Trigger Scenario The underwater state can arise via two paths: (1) significant user redemptions reduce rawAssets below currentLocked; (2) bad debt in a Moolah market directly reduces rawAssets without any user action. Initial: vault LP assets = 1,000 USDC, duration = 6h T=0 relayer flushes 9,000 broker interest: supply(9000) → rawAssets = 10,000 notify(9000) → currentLocked = 9,000 totalAssets() = 1,000 T=0-1h bad debt in a Moolah market, vault loses 3,000:

$\text{rawAssets} = 10,000 - 3,000 = 7,000$

$T=1h \quad \text{currentLocked} = 9,000 \times (6h-1h)/6h = 7,500$
 $\text{rawAssets} = 7,000 < \text{currentLocked} = 7,500$
 $\text{totalAssets}() = 0$

relayer attempts to flush 200 new interest:
 $\text{supply}(200) \rightarrow \text{rawAssets} = 7,200$
 $\text{totalAssets}() = \max(7,200 - 7,500, 0) = 0$
check: $200 > 0 \rightarrow \text{revert}$
entire transaction reverts, interest not delivered

Impact

All operations routed through `supplyToVault` are blocked while the vault is underwater, ordered by severity:

① Liquidations blocked (most severe)

LendingBroker's liquidation calls `_supplyToMoolahVault(interestToBroker)` after `MOOLAH.liquidate` completes. If L-01 triggers, the entire liquidation rolls back:

```
JavaScript
// LendingBroker - liquidate
MOOLAH.liquidate(...); // ← succeeds: collateral
seized, debt cleared
_supplyToMoolahVault(interestToBroker); // ← revert →
MOOLAH.liquidate also reverted
// unhealthy position remains open, bad debt continues to
accumulate
```

During market stress, mass redemptions and liquidation demand (collateral price drops) are positively correlated, making concurrent failure the most dangerous scenario.

② **Bad debt recovery interrupted:** CreditBroker's bad debt principal recovery routes through the same path; failed recovery leaves bad debt positions open.

③ **Penalty payments fail:** After revert, borrowers retrying later face longer overdue periods and potentially higher penalties.

④ **Interest delivery delayed:** Vault LPs lose earned yield for up to `MAX_DURATION` = 3 days.

Trigger probability is low, but when triggered the impact spans liquidation safety, bad debt governance, borrower welfare, and LP yield.

Recommendation

Apply graceful degradation inside `BrokerInterestLockBuffer.notifyBrokerInterest` — return silently when the vault is underwater rather than reverting. The flush proceeds without smoothing but does not block upstream callers:

JavaScript

```
function notifyBrokerInterest(uint256 amount) external override
onlyRole(RELAYER) {
    uint256 maxLockable = IERC4626(vault).totalAssets();
    // vault underwater: return silently - this flush is
    unsmoothed but liquidations are not blocked
    if (maxLockable == 0) return;
    uint256 effectiveAmount = amount > maxLockable ? maxLockable :
amount;
    uint256 newLocked = currentLocked() + effectiveAmount;
    if (newLocked > type(uint128).max) revert LockedOverflow();
    lockedAmount = uint128(newLocked);
    lastUpdate = uint64(block.timestamp);
    emit BrokerInterestNotified(msg.sender, effectiveAmount,
newLocked, block.timestamp + duration);
}
```

Fixed in commit [9e3da3d](#). Removed the amount > vault.totalAssets() cap check from notifyBrokerInterest entirely, eliminating the DOS vector when vault is underwater:

Status

JavaScript

```
if (amount > IERC4626(vault).totalAssets()) revert
AmountExceedsVaultAssets();
```

[L01] Removing a Non-Empty Buffer Causes an Immediate NAV Jump

Contract MoolahVault.sol

Severity Low

Description

The LockBufferNotEmpty check in setLockBuffer only validates that the new buffer is empty. It never checks whether the current (old) buffer still holds a residual locked amount:

JavaScript

```
// MoolahVault - setLockBuffer
function setLockBuffer(address buf) external onlyRole(MANAGER) {
    if (buf != address(0)) {
        if (IBrokerInterestLockBuffer(buf).vault() !=
address(this) || ...) revert LockBufferMismatch();
        if (IBrokerInterestLockBuffer(buf).currentLocked() != 0)
revert LockBufferNotEmpty();
        // ↑ checks new buffer only - old buffer is never checked
    }
    // buf == address(0) skips all checks entirely
```

```

        _updateLastTotalAssets(_accrueFee());
        lockBuffer = buf;
    }

```

When the current buffer still holds a residual locked amount, both of the following operations cause an immediate NAV jump:

- Detach ($A \rightarrow \text{address}(0)$): removing the buffer causes `totalAssets()` to stop deducting `currentLocked()`, instantly inflating NAV by that amount
- Replace ($A \rightarrow B$): switching to a new buffer version produces the same jump since the old buffer's locked amount is similarly ignored

Since `setLockBuffer` executes through a `TimeLock`, the pending proposal is publicly visible on-chain. An attacker can deposit while NAV is suppressed and redeem after the operation executes at the inflated NAV, capturing yield that belongs to existing LPs. The replace scenario is more operationally likely (e.g., upgrading to a new buffer version).

Impact

Both operations cause the same NAV discontinuity. $\text{Profit} \approx \text{deposit} \times \text{currentLocked} / \text{totalAssets}$ — typically small for large vaults but more significant for smaller ones.

Check the old buffer unconditionally, regardless of whether the operation is a detach or replace:

JavaScript

```

function setLockBuffer(address buf) external onlyRole(MANAGER) {
    address cur = lockBuffer;
    if (cur != address(0) &&
        IBrokerInterestLockBuffer(cur).currentLocked() != 0) {
        revert ErrorsLib.LockBufferNotEmpty();
    }
    if (buf != address(0)) {
        if (IBrokerInterestLockBuffer(buf).vault() !=
            address(this) || ...) revert LockBufferMismatch();
        if (IBrokerInterestLockBuffer(buf).currentLocked() != 0)
            revert LockBufferNotEmpty();
    }
    _updateLastTotalAssets(_accrueFee());
    lockBuffer = buf;
    emit EventsLib.SetLockBuffer(buf);
}

```

Recommendation

Status

Fixed in commit [f0ce6a6](#). Added old-buffer check to `setLockBuffer` covering both detach ($A \rightarrow 0$) and replace ($A \rightarrow B$):

JavaScript

```
address cur = lockBuffer;  
if (cur != address(0) &&  
IBrokerInterestLockBuffer(cur).currentLocked() != 0)  
    revert ErrorsLib.LockBufferNotEmpty();
```

[L02] Deflated Share Price During Lock Period Allows New Depositors to Dilute Existing LPs' Earned Interest

Contract MoolahVault.sol

Severity Low

Description

This is an inherent trade-off of the buffer mechanism rather than a design flaw. The BrokerInterestLockBuffer deflates totalAssets() by subtracting currentLocked(), which reduces the reported share price. While this prevents same-block sandwich attacks (depositor enters and exits at the same deflated price, capturing nothing), it creates an adverse side-effect for existing LPs when legitimate new deposits occur during the lock window.

Because totalAssets() is artificially low, new depositors receive more shares per asset than the true NAV justifies. As the locked interest linearly unlocks, totalAssets() gradually rises — and this unlocked value accrues pro-rata to all current shareholders, including the new entrants who purchased shares at the deflated price. The net effect is that existing LPs' earned broker interest is diluted by new deposits made during the lock period.

Description

This is the deposit-side complement to the withdrawal-side loss: withdrawers forfeit locked interest, while new depositors benefit from the unlocking interest proportionally along with existing LPs.

JavaScript

```
//MoolahVault.sol:453-464 - Deposit mints shares using the  
deflated totalAssets:
```

```
function deposit(uint256 assets, address receiver) public override  
returns (uint256 shares) {  
    require(isWhiteList(receiver), ErrorsLib.NotWhiteList());  
    uint256 newTotalAssets = _accrueFee(); // totalAssets() -  
currentLocked()
```

```
    lastTotalAssets = newTotalAssets;
```

```

        shares = _convertToSharesWithTotals(assets, totalSupply(),
newTotalAssets, Math.Rounding.Floor);
        //      ↑ newTotalAssets is deflated → denominator is smaller →
more shares minted

        _deposit(_msgSender(), receiver, assets, shares);
    }

//MoolahVault.sol:517-526 – The root cause: totalAssets()
underreports true NAV:

function totalAssets() public view override returns (uint256 assets)
{
    for (uint256 i; i < withdrawQueue.length; ++i) {
        assets +=
MOOLAH.expectedSupplyAssets(_marketParams(withdrawQueue[i]),
address(this));
    }
    address buf = lockBuffer;
    if (buf != address(0)) {
        uint256 locked =
IBrokerInterestLockBuffer(buf).currentLocked();
        assets = assets > locked ? assets - locked : 0;
    }
}

//MoolahVault.sol:596-602 – Share conversion where the deflated
denominator inflates minted shares:

function _convertToSharesWithTotals(
    uint256 assets,
    uint256 newTotalSupply,
    uint256 newTotalAssets, // deflated by currentLocked()
    Math.Rounding rounding
) internal view returns (uint256) {
    return assets.mulDiv(newTotalSupply + 10 ** _decimalsOffset(),
newTotalAssets + 1, rounding);
    //
smaller denominator → larger result
}

```

Impact

Example: Vault has 1,000 real assets, 1,000 shares (1:1). A 100-asset broker flush is locked (duration = 1 day).

	True NAV	Reported(totalAssets())	Share price
Before flush	1,000	1,000	1.00
After flush (t=0)	1,100	1,000	1.00 (deflated)
After unlock (t=1d)	1,100	1,100	1.10

	A new user deposits 100 assets at $t=0$, receiving -100 shares (at deflated price 1.00) instead of -91 shares (at true price 1.10). After full unlock: Without new deposit: existing 1,000 shares claim 1,100 assets $\rightarrow$ 1.10 per share With new deposit: 1,100 shares claim 1,200 assets $\rightarrow$ -1.091 per share Existing LPs' share price is 1.091 instead of 1.10 – the new depositor captured -9 assets of the existing LPs' earned broker interest. The new depositor's 100 shares are worth -109.1 (a -9.1% instant gain on the locked portion) despite contributing nothing to the interest generation. With the combine-and-reset mechanism keeping locked amounts persistently elevated under frequent flushes, this dilution becomes a continuous condition rather than a transient window.
Recommendation	At minimum, document this behavior clearly so users understand they should avoid depositing during active lock windows if they want to avoid diluting existing LPs' earned interest. Complex per-deposit checkpoint mechanisms are not recommended as they would significantly increase contract complexity and break standard ERC4626 semantics.
Status	Open. The deflated NAV during the lock window is a deliberate consequence of the sandwich-prevention mechanism: suppressing <code>totalAssets()</code> is what makes same-block front-run-deposit / back-run-redeem attacks unprofitable. Eliminating the deposit-side dilution would require per-entry share checkpointing, which breaks standard ERC4626 semantics and adds significant complexity. The trade-off is accepted as the cost of the anti-sandwich protection.

[L03] Buffer-Locking Mechanism Causes Withdrawing LPs to Forfeit Pro-Rata Share of Broker Interest	
Contract	MoolahVault.sol
Severity	Low
Description	Description This is the withdrawal-side complement to L02, and similarly reflects an inherent trade-off of the buffer mechanism. When broker interest is flushed into the vault via <code>BrokerInterestRelayer.supplyToVault()</code>, real assets are supplied to Moolah markets immediately, but the same amount is atomically locked in the <code>BrokerInterestLockBuffer</code>. The vault's <code>totalAssets()</code> subtracts <code>currentLocked()</code> from the true underlying balance, reporting a deflated NAV. Since all withdrawal and redemption paths (<code>_withdrawInternal</code>, <code>_redeemInternal</code>) derive share-to-asset conversion from <code>totalAssets()</code>, any LP who withdraws during an active lock period receives fewer assets than their true pro-rata entitlement. The forfeited value remains in the vault and gradually unlocks to benefit remaining LPs – constituting a wealth transfer from exiting users to holders.

The problem is compounded by the "combine-and-reset" design in `notifyBrokerInterest()`: each new flush adds to the remaining locked balance and restarts the decay clock. If flushes occur frequently relative to the duration (1h-3 days), the locked amount stays persistently elevated, making the underpricing semi-permanent for any user who does not wait out the full decay window.

JavaScript

`//MoolahVault.sol:517-526 - totalAssets() subtracts the locked amount:`

```
function totalAssets() public view override returns (uint256 assets)
{
    for (uint256 i; i < withdrawQueue.length; ++i) {
        assets +=
MOOLAH.expectedSupplyAssets(_marketParams(withdrawQueue[i]),
address(this));
    }
    address buf = lockBuffer;
    if (buf != address(0)) {
        uint256 locked =
IBrokerInterestLockBuffer(buf).currentLocked();
        assets = assets > locked ? assets - locked : 0;
    }
}
//MoolahVault.sol:662-678 - Redemption uses the deflated
totalAssets:
```

```
function _redeemInternal(
    uint256 shares,
    address receiver,
    address owner,
    address sender
) private returns (uint256 assets) {
    uint256 newTotalAssets = _accrueFee(); // calls totalAssets()
which subtracts currentLocked()

    assets = _convertToAssetsWithTotals(shares, totalSupply(),
newTotalAssets, Math.Rounding.Floor);

    _updateLastTotalAssets(newTotalAssets.zeroFloorSub(assets));
    _withdraw(sender, receiver, owner, assets, shares);
}
//BrokerInterestLockBuffer.sol:71-83 - Combine-and-reset extends the
lock window on every flush:
```

```
function notifyBrokerInterest(uint256 amount) external override
onlyRole(RELAYER) {
```

```

        if (amount > IERC4626(vault).totalAssets()) revert
AmountExceedsVaultAssets();

        uint256 newLocked = currentLocked() + amount; // stacks on
remaining locked
        if (newLocked > type(uint128).max) revert LockedOverflow();

        lockedAmount = uint128(newLocked);
        lastUpdate = uint64(block.timestamp); // resets the
clock for ALL locked value
        ...
    }

```

Impact

Legitimate vault LPs who withdraw or redeem during any active lock period receive fewer assets than their true pro-rata share. The deficit equals the user's proportional claim on the still-locked broker interest.

Example: A vault holds 1,000 real assets with 1,000 shares. A 100-asset broker flush is locked with a 1-day duration. A user holding 100 shares (10%) redeems immediately and receives -100 assets instead of their true entitlement of -110 assets. The -10-asset shortfall remains in the vault and unlocks to the benefit of remaining LPs.

With frequent broker flushes, the combine-and-reset mechanism keeps the locked balance persistently elevated, meaning the underpricing is not a brief transient but an ongoing condition that penalizes any exit during normal vault operation.

Recommendation

Consider tracking each user's "entry snapshot" of the locked amount so that withdrawals only exclude interest accrued after their deposit. Alternatively, apply the lock discount only to shares minted after the most recent flush rather than to totalAssets() globally — this preserves the anti-sandwich property while ensuring pre-existing LPs can exit at fair value. At minimum, document the trade-off clearly so users understand they should wait for the full unlock window before withdrawing after a broker flush event.

Status

Open. The withdrawal-side underpricing is the symmetric counterpart to L02 and stems from the same root cause. LPs who exit during an active lock window receive fewer assets than their true pro-rata entitlement; the shortfall unlocks to remaining holders over the decay period. A full fix would require pricing exits off raw assets rather than buffer-adjusted totalAssets(), which is an architectural

change outside the scope of this fix. Users are advised to wait for the lock window to expire before redeeming.

[L04] Underwater State Forces totalAssets to Zero and Causes Redeem to Return Near-Zero Assets

Contract MoolahVault.sol

Severity Low

Description

When `currentLocked > rawAssets` (underwater), `totalAssets()` is clamped to 0. `redeem / withdraw` price shares off this deflated `totalAssets()`. Exiting LPs burn shares but receive -0 assets, though raw assets remain on-chain. Forfeited value unlocks over time to remaining LPs (wealth transfer from exiters to holders).

Trigger (example):

T=0: `rawAssets=10000`, `currentLocked=9000`, `totalAssets=1000`

T=1h: users redeem -1000 visible → `rawAssets≈1000`, `currentLocked≈7500`, `totalAssets=0`

User with 100 shares (10%) redeems → gets -0 instead of -100

Related Code

Root cause: `totalAssets()` clamped to 0

Description

JavaScript

```
function totalAssets() public view override returns (uint256
assets) {
    // ...
    if (buf != address(0)) {
        uint256 locked =
IBrokerInterestLockBuffer(buf).currentLocked();
        assets = assets > locked ? assets - locked : 0; // ←
underwater → 0
    }
}
```

consequence: `redeem` uses that 0 for pricing

JavaScript

```
uint256 newTotalAssets = _accrueFee(); // == 0 when underwater
assets = _convertToAssetsWithTotals(shares, totalSupply(),
newTotalAssets, Math.Rounding.Floor);
```

	Impact • Low probability (large flush + heavy redemption of visible assets). • High severity per affected user: near-total loss on exit. • Funds not destroyed; accrue to remaining LPs until lock decays (up to 3 days). • Relay try/catch (M01 fix) does not fix this.
Recommendation	1. Governance escape: detach buffer or pause redemptions when underwater. 2. Or: price exits off raw assets / cap lock deduction without zero-flooring totalAssets(). 3. At minimum: document — do not redeem when currentLocked > rawAssets.
Status	Open. M01's fix (removing the amount > vault.totalAssets() cap check) does not resolve this issue. When vault is underwater, the relayer can now flush freely, but totalAssets() still clamps to 0 — new supply increases rawAssets by the flush amount while currentLocked increases by the same amount, leaving totalAssets() unchanged. Users who redeem while the vault is underwater still receive near-zero assets. Full resolution requires pricing exits off raw assets rather than buffer-adjusted totalAssets(), or a governance mechanism to pause redemptions when underwater.

[I01] Deploy Script Does Not Revoke Deployer's Temporary Roles, and the Referenced Follow-up Script Does Not Cover the Buffer

Contract	script/utils/deploy_brokerInterestLockBuffer.s.sol, script/broker/renounceRoles.s.sol
Severity	Informational
Description	Description The deploy script grants roles to their final holders but does not revoke the deployer's temporary roles, relying instead on a comment that references renounceRoles.s.sol as a follow-up: <pre> JavaScript buffer.grantRole(MANAGER, manager); buffer.grantRole(DEFAULT_ADMIN_ROLE, timelock); // Deployer's temporary roles should be revoked in a follow-up tx (matches the // pattern in script/broker/renounceRoles.s.sol). vm.stopBroadcast(); </pre> However, renounceRoles.s.sol only covers broker and relayer contracts — the buffer is absent entirely:

JavaScript

```
address[] brokers = [0x305a..., 0x6B01..., ...]; // 7 broker
addresses
address[] replayers; // relayer
list (empty)
// ← no buffer contract
```

The follow-up operation described in the comment does not exist for the buffer. After deployment, the deployer permanently retains:

- DEFAULT_ADMIN_ROLE: can upgrade the buffer implementation
- MANAGER: can modify the unlock duration

This contrasts with the project's established pattern (e.g. `deploy_moolahVaultTransferRole.sol`), where role handoff completes grant and revoke atomically within a single transaction, with no dependency on a separate follow-up.

Impact

The deployer EOA retains elevated privileges indefinitely. If the key is compromised, an attacker could upgrade the buffer to a malicious implementation or reset the unlock clock via `setDuration`, directly affecting vault LP NAV.

Revoke the deployer's roles at the end of the deploy script, consistent with the project's existing pattern:

Recommendation

JavaScript

```
buffer.grantRole(MANAGER, manager);
buffer.grantRole(DEFAULT_ADMIN_ROLE, timelock);
buffer.revokeRole(MANAGER, deployer);
buffer.revokeRole(DEFAULT_ADMIN_ROLE, deployer);
vm.stopBroadcast();
```

Fixed in commit [f0ce6a6](#). Deploy script now revokes deployer's temporary roles atomically in the same broadcast:

Status

JavaScript

```
buffer.revokeRole(MANAGER, deployer);
buffer.revokeRole(DEFAULT_ADMIN_ROLE, deployer);
```